Linux Complete Guide

Day 11 - Day 15

Package Management | Services | Monitoring | Backup | Networking

- Package Management (APT, YUM, DNF, RPM)
- Services and Daemons (systemd)
- System Information and Monitoring
- Backup and Archiving (tar, rsync)
- Linux Networking Basics

Prepared by Raman

Connect on LinkedIn

Contents

DAY 11: Package Management

Installing and Managing Software Packages

Theory

What is Package Management?
A package manager is a tool that automates the process of installing, upgrading, configuring, and removing software packages. Linux distributions use different package managers:

- **Debian/Ubuntu:** APT (Advanced Package Tool) with .deb packages
- **RHEL/CentOS/Fedora:** YUM/DNF with .rpm packages

Package Manager Types:

- **High-level:** apt, yum, dnf (resolve dependencies automatically)
- **Low-level:** dpkg, rpm (manual dependency handling)

Debian/Ubuntu Package Management

apt-get - Advanced Package Tool

Flags and Options

Main Commands:

`apt-get update`	Update package list from repositories
`apt-get upgrade`	Upgrade all installed packages
`apt-get dist-upgrade`	Upgrade with dependency changes
`apt-get install PKG`	Install a package
`apt-get remove PKG`	Remove package (keep config)
`apt-get purge PKG`	Remove package AND config files
`apt-get autoremove`	Remove unused dependencies
`apt-get clean`	Clear downloaded package cache
`apt-get autoclean`	Clear old package cache only

Common Options:

`-y`	Assume yes to all prompts (non-interactive)
`-q`	Quiet mode (less output)
`-f`	Fix broken dependencies
`-d`	Download only (do not install)
`-no-install-recommends`	Do not install recommended packages
`-reinstall`	Reinstall an already installed package

Examples

```
# Step 1: Always update package list first
$ sudo apt-get update

# Step 2: Upgrade all installed packages
$ sudo apt-get upgrade -y

# Full system upgrade (may add/remove packages)
$ sudo apt-get dist-upgrade -y

# Install a single package
$ sudo apt-get install nginx

# Install multiple packages
$ sudo apt-get install nginx mysql-server php

# Install specific version
$ sudo apt-get install nginx=1.18.0-0ubuntu1

# Install without recommended packages (smaller)
$ sudo apt-get install --no-install-recommends nginx

# Remove package (keep configuration files)
$ sudo apt-get remove nginx

# Remove package AND configuration files
$ sudo apt-get purge nginx

# Remove unused dependencies
$ sudo apt-get autoremove

# Fix broken packages
$ sudo apt-get install -f

# Clean package cache (free disk space)
$ sudo apt-get clean
$ sudo apt-get autoclean
```

apt-cache - Query Package Information

Flags and Options

`search PATTERN`	Search packages by name or description
`show PKG`	Show detailed package information
`showpkg PKG`	Show package dependencies
`depends PKG`	Show what package depends on
`rdepends PKG`	Show reverse dependencies
`policy PKG`	Show installation policy and versions
`stats`	Show cache statistics
`pkgnames`	List all available package names

Examples

```
# Search for packages containing "nginx"
$ apt-cache search nginx
nginx - small, powerful, scalable web/proxy server
nginx-common - small, powerful, scalable web/proxy server - common files
nginx-core - nginx web/proxy server (standard version)

# Search with regular expression
$ apt-cache search "^nginx"

# Show detailed package information
$ apt-cache show nginx
Package: nginx
Version: 1.18.0-0ubuntu1
Priority: optional
Section: web
Maintainer: Ubuntu Developers
Installed-Size: 39
Depends: nginx-core | nginx-full | nginx-light | nginx-extras
...

# Show package dependencies
$ apt-cache depends nginx
nginx
  Depends: nginx-core
  Depends: nginx-full
  Depends: nginx-light

# Show which version will be installed
$ apt-cache policy nginx
nginx:
  Installed: 1.18.0-0ubuntu1
  Candidate: 1.18.0-0ubuntu1
  Version table:
 *** 1.18.0-0ubuntu1 500
        500 http://archive.ubuntu.com/ubuntu focal/main amd64

# List all available packages
$ apt-cache pkgnames | wc -l
```

dpkg - Debian Package Manager (Low-Level)

Flags and Options

Installation and Removal:

`-i file.deb`	Install a .deb package file
`-r PKG`	Remove package (keep config files)
`-P PKG`	Purge package (remove config too)
`-configure -a`	Configure all unpacked packages

Query Commands:

`-l`	List all installed packages
`-l PATTERN`	List packages matching pattern
`-L PKG`	List files installed by package
`-s PKG`	Show package status/info
`-S FILE`	Find which package owns a file
`-c file.deb`	List contents of .deb file
`-I file.deb`	Show info about .deb file

Examples

```
# Install a downloaded .deb file
$ sudo dpkg -i google-chrome-stable_current_amd64.deb

# If dependencies are missing, fix with:
$ sudo apt-get install -f

# Remove a package (keep config)
$ sudo dpkg -r google-chrome-stable

# Purge a package (remove config too)
$ sudo dpkg -P google-chrome-stable

# List all installed packages
$ dpkg -l
$ dpkg -l | wc -l          # Count installed packages

# List packages matching pattern
$ dpkg -l | grep nginx
ii  nginx  1.18.0-0ubuntu1  amd64  small, powerful web server

# List files installed by a package
$ dpkg -L nginx
/etc/nginx
/etc/nginx/nginx.conf
/usr/sbin/nginx
/var/log/nginx
...

# Find which package owns a file
$ dpkg -S /usr/bin/vim
vim: /usr/bin/vim

$ dpkg -S /etc/nginx/nginx.conf
nginx-common: /etc/nginx/nginx.conf
```

```
# Show package status and info
$ dpkg -s nginx
Package: nginx
Status: install ok installed
Priority: optional
Section: web
Installed-Size: 39
...

# List contents of .deb file (before installing)
$ dpkg -c package.deb

# Show info about .deb file
$ dpkg -I package.deb
```

CentOS/RHEL Package Management

yum - Yellowdog Updater Modified

Flags and Options

Main Commands:

`yum install PKG`	Install a package
`yum update`	Update all packages
`yum update PKG`	Update specific package
`yum remove PKG`	Remove a package
`yum search TERM`	Search for packages
`yum info PKG`	Show package information
`yum list installed`	List installed packages
`yum list available`	List available packages
`yum clean all`	Clean cache
`yum repolist`	List enabled repositories
`yum provides FILE`	Find package providing file
`yum history`	Show transaction history
`yum groupinstall`	Install package group

Common Options:

`-y`	Assume yes to all prompts
`-q`	Quiet mode
`-nogpgcheck`	Skip GPG signature verification
`-enablerepo=`	Temporarily enable a repository
`-disablerepo=`	Temporarily disable a repository

Examples

```
# Install a package
$ sudo yum install httpd

# Install with automatic yes
$ sudo yum install -y httpd

# Install multiple packages
$ sudo yum install -y httpd mariadb-server php

# Update all packages
$ sudo yum update -y

# Update specific package
$ sudo yum update httpd

# Remove a package
$ sudo yum remove httpd

# Search for packages
$ yum search nginx
$ yum search "web server"

# Show package information
$ yum info httpd
```

```
# List installed packages
$ yum list installed
$ yum list installed | grep http

# List all available packages
$ yum list available

# Find which package provides a file
$ yum provides /usr/sbin/httpd
$ yum provides "*/nginx"

# Clean cache
$ sudo yum clean all

# List repositories
$ yum repolist
$ yum repolist all    # Include disabled

# View transaction history
$ yum history
$ yum history info 5        # Show transaction 5
$ sudo yum history undo 5   # Undo transaction 5

# Install package group
$ sudo yum groupinstall "Development Tools"
$ yum grouplist
```

dnf - Dandified YUM (Modern Replacement)

Theory

DNF is the next-generation replacement for YUM, used in:

- Fedora 22+
- CentOS 8+ / Rocky Linux / AlmaLinux
- RHEL 8+

DNF has better dependency resolution, faster performance, and uses less memory. Most commands are identical to YUM.

Examples

```
# All yum commands work with dnf
$ sudo dnf install httpd
$ sudo dnf update
$ sudo dnf remove httpd
$ dnf search nginx
$ dnf info httpd
$ dnf list installed

# DNF-specific features
$ dnf check-update            # Check for available updates
$ dnf repoquery --list PKG    # List files in package
$ dnf autoremove              # Remove unneeded packages
```

```
$ dnf module list            # List available modules
$ sudo dnf module enable php:7.4
$ dnf history
$ sudo dnf history undo 5
```

rpm - RPM Package Manager (Low-Level)

Flags and Options

Installation:

`-i PKG.rpm`	Install a package
`-U PKG.rpm`	Upgrade (install if not present)
`-F PKG.rpm`	Freshen (upgrade only if installed)
`-v`	Verbose output
`-h`	Show hash marks (progress)
`-nodeps`	Skip dependency checks (dangerous!)
`-force`	Force installation

Removal:

`-e PKG`	Erase (uninstall) a package

Query:

`-q PKG`	Query if package is installed
`-qa`	Query all installed packages
`-qi PKG`	Query package info
`-ql PKG`	Query list of files in package
`-qf FILE`	Query which package owns file
`-qc PKG`	Query config files of package
`-qd PKG`	Query documentation files
`-qR PKG`	Query package requirements
`-V PKG`	Verify package integrity

Examples

```
# Install an RPM file (verbose with progress)
$ sudo rpm -ivh package.rpm

# Upgrade a package
$ sudo rpm -Uvh package.rpm

# Remove a package
$ sudo rpm -e package_name

# Query if package is installed
$ rpm -q httpd
httpd-2.4.6-97.el7.centos.x86_64

# List ALL installed packages
$ rpm -qa
$ rpm -qa | wc -l
$ rpm -qa | grep httpd
```

```
# Show package information
$ rpm -qi httpd

# List files installed by package
$ rpm -ql httpd
/etc/httpd
/etc/httpd/conf
/etc/httpd/conf/httpd.conf
/usr/sbin/httpd
...

# Find which package owns a file
$ rpm -qf /usr/sbin/httpd
httpd-2.4.6-97.el7.centos.x86_64

# List config files
$ rpm -qc httpd

# List documentation files
$ rpm -qd httpd

# Verify package (check if files modified)
$ rpm -V httpd
S.5....T.  c /etc/httpd/conf/httpd.conf
# S=Size, 5=MD5, T=Time changed, c=config file
```

Pro Tip

Low-level vs High-level Package Managers:

- `dpkg` and `rpm` do NOT resolve dependencies
- Always prefer `apt/apt-get` over `dpkg`
- Always prefer `yum/dnf` over `rpm`
- Use low-level tools for querying or manual .deb/.rpm installation

Hands-On Practice

Day 11 Exercises:
Exercise 1: Update and Install (Ubuntu)

```
$ sudo apt-get update
$ apt-cache search htop
$ sudo apt-get install htop
$ htop --version
$ apt-cache show htop
$ dpkg -L htop
```

Exercise 2: Query Packages

```
# Ubuntu
$ dpkg -l | grep -i apache
$ dpkg -S /usr/bin/wget

# CentOS
$ rpm -qa | grep -i http
$ rpm -qf /usr/bin/wget
```

Exercise 3: Clean Up

```
$ sudo apt-get remove htop
$ sudo apt-get autoremove
$ sudo apt-get clean
```

Interview Question

Q: What is the difference between apt-get remove and apt-get purge?
`remove` uninstalls the package but keeps configuration files for potential reinstallation. `purge` removes the package AND all its configuration files completely.

Q: What is the difference between yum and dnf?
DNF is the next-generation version of YUM with better dependency resolution, faster performance, and cleaner codebase. Most commands are identical. DNF is the default on Fedora, CentOS 8+, and RHEL 8+.

Q: How do you find which package provides a specific file?
On Debian/Ubuntu: `dpkg -S /path/to/file`
On RHEL/CentOS: `rpm -qf /path/to/file` or `yum provides /path/to/file`

DAY 12: Services and Daemons

Managing Background Services with systemd

Theory

What are Services and Daemons?

- **Daemon:** A background process that runs continuously (usually ends with 'd' - sshd, httpd, mysqld)
- **Service:** A managed daemon with start/stop/status controls
- **systemd:** The modern init system that manages services, logging, and system state
- **Unit files:** Configuration files in `/etc/systemd/system/` or `/lib/systemd/system/`

Service States:

- **active (running):** Service is currently running
- **inactive (dead):** Service is stopped
- **enabled:** Service will start automatically at boot
- **disabled:** Service will NOT start at boot
- **masked:** Service is completely blocked from starting

systemctl - Systemd Control Command

Flags and Options

Service Control Commands:

`systemctl start SERVICE`	Start a service immediately
`systemctl stop SERVICE`	Stop a service immediately
`systemctl restart SERVICE`	Stop then start service
`systemctl reload SERVICE`	Reload config (no restart)
`systemctl status SERVICE`	Show service status
`systemctl enable SERVICE`	Enable start at boot
`systemctl disable SERVICE`	Disable start at boot
`systemctl enable -now SERVICE`	Enable AND start
`systemctl is-active SERVICE`	Check if running
`systemctl is-enabled SERVICE`	Check if enabled
`systemctl mask SERVICE`	Completely disable
`systemctl unmask SERVICE`	Remove mask

Listing and Information Commands:

`systemctl list-units`	List active units
`systemctl list-units -type=service`	List services only
`systemctl list-units -failed`	List failed services
`systemctl list-unit-files`	List all unit files
`systemctl cat SERVICE`	Show unit file contents
`systemctl show SERVICE`	Show all properties
`systemctl daemon-reload`	Reload systemd config

System Commands:

`systemctl reboot`	Reboot the system
`systemctl poweroff`	Shutdown the system
`systemctl suspend`	Suspend the system
`systemctl hibernate`	Hibernate the system
`systemctl get-default`	Show default target
`systemctl set-default`	Set default target

Examples

```
# Check service status (most common command)
$ sudo systemctl status nginx
  nginx.service - A high performance web server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
   Active: active (running) since Mon 2024-01-15 10:00:00 UTC
  Process: 1234 ExecStart=/usr/sbin/nginx (code=exited, status=0)
 Main PID: 1235 (nginx)
    Tasks: 2 (limit: 4915)
   Memory: 3.5M
   CGroup: /system.slice/nginx.service
           +-1235 nginx: master process /usr/sbin/nginx
           +-1236 nginx: worker process

# Start a service
$ sudo systemctl start nginx

# Stop a service
$ sudo systemctl stop nginx

# Restart a service (stop + start)
$ sudo systemctl restart nginx

# Reload configuration without stopping
$ sudo systemctl reload nginx

# Reload OR restart (reload if supported)
$ sudo systemctl reload-or-restart nginx

# Enable service to start at boot
$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service

# Disable service from starting at boot
$ sudo systemctl disable nginx
Removed /etc/systemd/system/multi-user.target.wants/nginx.service

# Enable AND start in one command
$ sudo systemctl enable --now nginx

# Check if service is running
$ systemctl is-active nginx
active

# Check if service is enabled
$ systemctl is-enabled nginx
enabled
```

```
# List all running services
$ systemctl list-units --type=service

# List all services (running and not)
$ systemctl list-units --type=service --all

# List failed services
$ systemctl list-units --failed

# List all service unit files
$ systemctl list-unit-files --type=service

# Show unit file content
$ systemctl cat nginx.service

# Show all service properties
$ systemctl show nginx

# Mask service (prevent starting by any means)
$ sudo systemctl mask nginx

# Unmask service
$ sudo systemctl unmask nginx

# Reload systemd after editing unit files
$ sudo systemctl daemon-reload
```

Pro Tip

Understanding systemctl status output:

```
$ systemctl status sshd
* sshd.service - OpenSSH server daemon
   Loaded: loaded (/lib/systemd/system/sshd.service; enabled)
   Active: active (running) since Mon 2024-01-15 08:00:00 UTC

# Key Information:
# Loaded: Path to unit file + enabled/disabled status
# Active: Current state (running/dead/failed)
# Main PID: Process ID of main service process
# Tasks: Number of tasks/threads
# Memory: Memory usage
# CGroup: Control group hierarchy
```

enable vs start:

- `start` = Start service NOW (temporary, lost on reboot)
- `enable` = Start service at BOOT (permanent)
- Use `enable -now` to do BOTH

service - Legacy SysVinit Command

Theory

The `service` command is from the older SysVinit system. On modern systems with systemd, it acts as a wrapper that calls `systemctl`. Init scripts are stored in `/etc/init.d/`.

Flags and Options

`service NAME start`	Start the service
`service NAME stop`	Stop the service
`service NAME restart`	Restart the service
`service NAME reload`	Reload configuration
`service NAME status`	Show service status
`service -status-all`	Show status of all services

Examples

```
# Start a service
$ sudo service nginx start

# Stop a service
$ sudo service nginx stop

# Restart a service
$ sudo service nginx restart

# Check service status
$ sudo service nginx status

# List all services and their status
$ service --status-all
 [ + ]  nginx            # Running
 [ - ]  apache2          # Stopped
 [ ? ]  networking       # Unknown/no status command

# Legend:
# [ + ] = Service is running
# [ - ] = Service is stopped
# [ ? ] = No status command available
```

Hands-On Practice

Day 12 Exercises:
Exercise 1: Install and Manage Web Server

```
# Install nginx
$ sudo apt-get install nginx    # Ubuntu
$ sudo yum install nginx        # CentOS

# Check status
$ sudo systemctl status nginx

# Start/stop/restart
$ sudo systemctl stop nginx
$ systemctl is-active nginx
```

```
$ sudo systemctl start nginx
$ systemctl is-active nginx

# Enable at boot
$ sudo systemctl enable nginx
$ systemctl is-enabled nginx
```

Exercise 2: Explore Running Services

```
$ systemctl list-units --type=service | head -20
$ systemctl list-units --failed
$ systemctl cat sshd.service
```

Exercise 3: Compare systemctl and service

```
$ sudo systemctl status nginx
$ sudo service nginx status
# Both work on modern systems!
```

Interview Question

Q: What is the difference between systemctl enable and systemctl start?
`start` starts the service immediately for the current session only. `enable` creates symbolic links so the service starts automatically at boot, but doesn't start it now. Use `enable -now` to do both.

Q: How do you check why a service failed to start?
Use `systemctl status service-name` to see recent logs, or `journalctl -u service-name` for full logs. Check `journalctl -xe` for the most recent errors with explanations.

Q: What is the difference between restart and reload?
`restart` stops and starts the service (brief downtime). `reload` tells the service to re-read its configuration without stopping (no downtime). Not all services support reload.

DAY 13: System Info and Monitoring

Hardware and System Information Commands

uname - System Information

Flags and Options

-a	Print ALL information
-s	Print kernel name (Linux)
-n	Print network hostname
-r	Print kernel release version
-v	Print kernel version
-m	Print machine hardware name (x86_64)
-p	Print processor type
-i	Print hardware platform
-o	Print operating system

Examples

```
# Show all system information
$ uname -a
Linux server01 5.15.0-91-generic #101-Ubuntu SMP x86_64 GNU/Linux

# Kernel name only
$ uname -s
Linux

# Kernel release/version (most commonly used)
$ uname -r
5.15.0-91-generic

# Machine architecture
$ uname -m
x86_64

# Operating system
$ uname -o
GNU/Linux

# Hostname
$ uname -n
server01
```

hostname - Display/Set System Hostname

Flags and Options

`-f, -fqdn`	Display Fully Qualified Domain Name
`-i`	Display IP address of hostname
`-I`	Display ALL IP addresses
`-s`	Display short hostname
`-d`	Display DNS domain name

Examples

```
# Display hostname
$ hostname
webserver01

# Display FQDN
$ hostname -f
webserver01.example.com

# Display all IP addresses
$ hostname -I
192.168.1.100 10.0.0.5 172.17.0.1

# Set hostname temporarily
$ sudo hostname newhostname

# Set hostname permanently (systemd)
$ sudo hostnamectl set-hostname newserver

# View hostnamectl info
$ hostnamectl
   Static hostname: server01
         Icon name: computer-vm
           Chassis: vm
        Machine ID: abc123...
           Boot ID: def456...
  Operating System: Ubuntu 22.04.3 LTS
            Kernel: Linux 5.15.0-91-generic
      Architecture: x86-64
```

uptime - System Uptime and Load Average

Examples

```
# Show uptime
$ uptime
 14:32:15 up 45 days, 3:22, 3 users, load average: 0.15, 0.20, 0.18

# Breakdown:
# 14:32:15       - Current time
# up 45 days     - System uptime
# 3 users        - Number of logged in users
# load average   - 1 min, 5 min, 15 min CPU load averages
```

```
# Pretty format
$ uptime -p
up 45 days, 3 hours, 22 minutes

# Since when (boot time)
$ uptime -s
2023-11-30 11:10:00
```

Pro Tip

Understanding Load Average:
Load average represents the average number of processes waiting for CPU time.

- Values should ideally be less than the number of CPU cores
- On a 4-core system: load < 4.0 is healthy
- Load > number of cores = system is overloaded
- Three values: 1-minute, 5-minute, 15-minute averages

dmesg - Kernel Ring Buffer Messages

Flags and Options

`-H`	Human-readable output (with colors)
`-T`	Display human-readable timestamps
`-w`	Wait for new messages (follow mode)
`-l LEVEL`	Filter by log level (err, warn, info)
`-c`	Clear ring buffer after reading
`-level=`	Filter by level (emerg,alert,crit,err,warn)

Examples

```
# View kernel messages
$ dmesg | less

# Human-readable with timestamps
$ dmesg -TH

# Filter by level
$ dmesg -l err
$ dmesg --level=err,warn

# Search for specific hardware
$ dmesg | grep -i usb
$ dmesg | grep -i eth
$ dmesg | grep -i error
$ dmesg | grep -i disk

# Follow new messages (like tail -f)
$ dmesg -w

# Show only last 20 messages
$ dmesg | tail -20
```

```
# Clear buffer (requires root)
$ sudo dmesg -c
```

free - Memory Usage

Flags and Options

`-h`	Human-readable (KB, MB, GB)
`-b`	Display in bytes
`-k`	Display in kilobytes (default)
`-m`	Display in megabytes
`-g`	Display in gigabytes
`-t`	Show total line
`-s N`	Update every N seconds
`-c N`	Display N times and exit
`-w`	Wide output (separate buffers/cache)

Examples

```
# Human-readable memory information
$ free -h
              total        used        free      shared  buff/cache
   available
Mem:           16Gi       4.2Gi       8.1Gi       512Mi       3.7Gi
          11Gi
Swap:         2.0Gi          0B       2.0Gi

# With totals
$ free -ht

# Wide output (more details)
$ free -hw

# Update every 2 seconds
$ free -h -s 2

# Display 5 times then exit
$ free -h -c 5 -s 1

# Column Meanings:
# total     - Total installed RAM
# used      - Currently used memory
# free      - Completely unused memory
# shared    - Memory used by tmpfs
# buff/cache - Buffer + cached data (can be freed)
# available  - Memory available for new applications
```

vmstat - Virtual Memory Statistics

Examples

```
# Basic virtual memory stats
$ vmstat
procs -----------memory---------- ---swap-- -----io---- -system-- ------
    cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy
     id wa st
 1  0      0 8456712 234567 3876543    0    0     5    10  100  200  2  1
      97  0  0

# Update every 2 seconds
$ vmstat 2

# Update every 2 seconds, 5 times
$ vmstat 2 5

# With timestamps
$ vmstat -t 2 5

# Show disk statistics
$ vmstat -d

# Show memory in MB
$ vmstat -S M

# Key columns:
# r  - Processes waiting for CPU
# b  - Processes in uninterruptible sleep
# swpd - Virtual memory used
# free - Idle memory
# si/so - Swap in/out from disk
# us - User CPU time
# sy - System/kernel CPU time
# id - Idle time
# wa - Wait I/O time
```

lscpu - CPU Information

Examples

```
$ lscpu
Architecture:            x86_64
CPU op-mode(s):          32-bit, 64-bit
Byte Order:              Little Endian
Address sizes:           46 bits physical, 48 bits virtual
CPU(s):                  4
On-line CPU(s) list:     0-3
Thread(s) per core:      2
Core(s) per socket:      2
Socket(s):               1
NUMA node(s):            1
Vendor ID:               GenuineIntel
```

```
CPU family:                6
Model:                     142
Model name:                Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz
Stepping:                  11
CPU MHz:                   1992.000
CPU max MHz:               4600.0000
CPU min MHz:               400.0000
BogoMIPS:                  3984.00
Virtualization:            VT-x
L1d cache:                 64 KiB
L1i cache:                 64 KiB
L2 cache:                  512 KiB
L3 cache:                  8 MiB

# Parse-friendly output
$ lscpu -p

# JSON output
$ lscpu -J
```

lsusb, lspci, lshw - Hardware Information

Examples

```
# List USB devices
$ lsusb
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation USB 3.0 root hub
Bus 001 Device 002: ID 8087:0a2b Intel Corp. Bluetooth wireless interface
Bus 001 Device 004: ID 046d:c52b Logitech Unifying Receiver

# Verbose USB info
$ lsusb -v
$ lsusb -t    # Tree view

# List PCI devices
$ lspci
00:00.0 Host bridge: Intel Corporation Device 3e34
00:02.0 VGA compatible controller: Intel Corporation Device 3ea0
00:1f.3 Audio device: Intel Corporation Cannon Point-LP High Definition
   Audio

# Verbose PCI
$ lspci -v
$ lspci -vv   # Very verbose

# Filter PCI devices
$ lspci | grep -i network
$ lspci | grep -i vga
$ lspci | grep -i audio

# Comprehensive hardware list (requires root)
$ sudo lshw

# Short summary
```

```
$ sudo lshw -short

# HTML report
$ sudo lshw -html > hardware.html

# Specific class
$ sudo lshw -class network
$ sudo lshw -class disk
$ sudo lshw -class memory
$ sudo lshw -class processor
```

Hands-On Practice

Day 13 Exercises:
Exercise 1: Gather System Information

```
$ uname -a
$ hostname -I
$ uptime
$ uptime -p
```

Exercise 2: Check Memory and CPU

```
$ free -h
$ vmstat 1 5
$ lscpu | head -20
```

Exercise 3: Hardware Discovery

```
$ lsusb
$ lspci | head -10
$ sudo lshw -short
```

Exercise 4: Check Kernel Messages

```
$ dmesg | tail -30
$ dmesg | grep -i error
$ dmesg | grep -i usb
```

Interview Question

Q: How do you check the Linux kernel version?
Use `uname -r` to see the kernel release version, or `uname -a` for all system information including kernel version.

Q: How do you check memory usage in Linux?
Use `free -h` for a human-readable summary of memory usage including RAM and swap. The "available" column shows memory that can be used by new applications.

Q: What is load average and what do the three numbers mean?
Load average represents the average number of processes waiting for CPU. The three numbers are averages over 1 minute, 5 minutes, and 15 minutes. Values below the number of CPU cores indicate a healthy system.

DAY 14: Backup and Archiving

Compressing and Archiving Files

tar - Tape Archive

Theory

`tar` creates archives (collections of files) and can optionally compress them. It's the most common backup tool in Linux.

Common File Extensions:

- `.tar` - Uncompressed archive
- `.tar.gz` or `.tgz` - Gzip compressed (most common)
- `.tar.bz2` - Bzip2 compressed (better compression)
- `.tar.xz` - XZ compressed (best compression)

Flags and Options

Main Operations:

`-c`	Create a new archive
`-x`	Extract files from archive
`-t`	List contents of archive
`-r`	Append files to archive
`-u`	Update (add newer files only)

Compression Options:

`-z`	Compress/decompress with gzip (.tar.gz)
`-j`	Compress/decompress with bzip2 (.tar.bz2)
`-J`	Compress/decompress with xz (.tar.xz)
`-a`	Auto-detect compression from extension

Common Options:

`-v`	Verbose (show files being processed)
`-f FILE`	Specify archive filename (MUST be last!)
`-C DIR`	Change to directory before operation
`-p`	Preserve permissions
`-exclude=`	Exclude files matching pattern
`-T FILE`	Read list of files from FILE
`-newer=DATE`	Only files newer than DATE

Examples

```
# ========== CREATING ARCHIVES ==========

# Create tar archive (no compression)
$ tar -cvf archive.tar folder/
$ tar -cvf archive.tar file1.txt file2.txt folder/

# Create GZIP compressed archive (.tar.gz) - MOST COMMON
$ tar -czvf backup.tar.gz folder/
```

```
# Create BZIP2 compressed archive (.tar.bz2)
$ tar -cjvf backup.tar.bz2 folder/

# Create XZ compressed archive (.tar.xz) - BEST COMPRESSION
$ tar -cJvf backup.tar.xz folder/

# Archive with full path preservation
$ tar -cvf archive.tar -C /home/user documents/

# Exclude files/directories
$ tar -czvf backup.tar.gz --exclude="*.log" --exclude="cache" folder/
$ tar -czvf backup.tar.gz --exclude-vcs folder/  # Exclude .git, .svn

# Archive files newer than a date
$ tar -czvf backup.tar.gz --newer="2024-01-01" folder/

# ========== LISTING CONTENTS ==========

# List contents without extracting
$ tar -tvf archive.tar
$ tar -tzvf backup.tar.gz

# ========== EXTRACTING ==========

# Extract to current directory
$ tar -xvf archive.tar
$ tar -xzvf backup.tar.gz
$ tar -xjvf backup.tar.bz2
$ tar -xJvf backup.tar.xz

# Extract to specific directory
$ tar -xzvf backup.tar.gz -C /tmp/restore/

# Extract specific file from archive
$ tar -xzvf backup.tar.gz path/to/specific/file.txt

# Extract with preserved permissions
$ tar -xpzvf backup.tar.gz

# ========== PRACTICAL EXAMPLES ==========

# Backup home directory
$ tar -czvf home_backup_$(date +%Y%m%d).tar.gz /home/user/

# Backup with exclusions
$ tar -czvf backup.tar.gz \
    --exclude="*.log" \
    --exclude="*.tmp" \
    --exclude=".cache" \
    /var/www/

# Incremental backup (files changed since last backup)
$ tar -czvf incremental.tar.gz --newer-mtime="2024-01-15" /data/
```

Pro Tip

Remember tar flags: Create/eXtract Ze Verbose File

- `-czvf` for creating .tar.gz
- `-xzvf` for extracting .tar.gz
- `-f` must be last (followed by filename)

zip / unzip - ZIP Format

Flags and Options

zip options:

`-r`	Recursive (include subdirectories)
`-e`	Encrypt with password
`-u`	Update (add new/changed files only)
`-d`	Delete files from archive
`-9`	Maximum compression
`-0`	No compression (store only)
`-x PATTERN`	Exclude files matching pattern

unzip options:

`-l`	List contents
`-t`	Test archive integrity
`-d DIR`	Extract to directory
`-o`	Overwrite without prompting
`-n`	Never overwrite existing files

Examples

```
# Create zip archive
$ zip archive.zip file1.txt file2.txt

# Create zip with directory (recursive)
$ zip -r archive.zip folder/

# Create encrypted zip
$ zip -e secure.zip sensitive_file.txt

# Maximum compression
$ zip -r9 archive.zip folder/

# Exclude files
$ zip -r archive.zip folder/ -x "*.log" -x "*.tmp"

# Update existing archive
$ zip -u archive.zip newfile.txt

# List contents
$ unzip -l archive.zip

# Test archive integrity
$ unzip -t archive.zip
```

```
# Extract archive
$ unzip archive.zip

# Extract to specific directory
$ unzip archive.zip -d /tmp/extracted/

# Extract without overwriting
$ unzip -n archive.zip
```

gzip / gunzip - GNU Zip Compression

Flags and Options

`-k`	Keep original file
`-d`	Decompress (same as gunzip)
`-c`	Write to stdout (keep original)
`-1` to `-9`	Compression level (1=fast, 9=best)
`-v`	Verbose
`-l`	List compression info
`-r`	Recursive (compress files in directory)

Examples

```
# Compress file (REPLACES original with .gz)
$ gzip file.txt
# Result: file.txt.gz (original deleted!)

# Keep original file
$ gzip -k file.txt
# Result: file.txt AND file.txt.gz

# Decompress
$ gunzip file.txt.gz
$ gzip -d file.txt.gz

# Best compression
$ gzip -9 file.txt

# Fastest compression
$ gzip -1 file.txt

# View compressed file without decompressing
$ zcat file.txt.gz
$ zless file.txt.gz
$ zgrep "pattern" file.txt.gz

# Show compression ratio
$ gzip -l file.txt.gz
         compressed        uncompressed  ratio uncompressed_name
               1234                5678  78.3% file.txt
```

bzip2 / bunzip2 - Better Compression

Examples

```
# Compress (replaces original with .bz2)
$ bzip2 file.txt

# Keep original
$ bzip2 -k file.txt

# Decompress
$ bunzip2 file.txt.bz2
$ bzip2 -d file.txt.bz2

# View without decompressing
$ bzcat file.txt.bz2
$ bzless file.txt.bz2
$ bzgrep "pattern" file.txt.bz2
```

xz / unxz - Best Compression Ratio

Examples

```
# Compress (replaces original with .xz)
$ xz file.txt

# Keep original
$ xz -k file.txt

# Decompress
$ unxz file.txt.xz
$ xz -d file.txt.xz

# View without decompressing
$ xzcat file.txt.xz
$ xzless file.txt.xz
$ xzgrep "pattern" file.txt.xz

# Extreme compression (slow but smallest)
$ xz -9e file.txt
```

Pro Tip

Compression Comparison:

Tool	Speed	Ratio	Extension
gzip	Fast	Good	.gz
bzip2	Slow	Better	.bz2
xz	Slowest	Best	.xz

For most use cases, `gzip` is the best balance of speed and compression.

rsync - Remote Sync

Theory

`rsync` is a powerful file synchronization tool that:

- Only transfers CHANGED portions of files (delta transfer)
- Can resume interrupted transfers
- Works locally or over SSH
- Preserves permissions, timestamps, and symbolic links
- Much faster than `cp` for incremental backups

Flags and Options

`-a`	Archive mode (preserves everything, recursive)
`-v`	Verbose output
`-z`	Compress during transfer
`-h`	Human-readable output
`-P`	Show progress + keep partial transfers
`-progress`	Show transfer progress
`-r`	Recursive
`-n, -dry-run`	Test without making changes
`-delete`	Delete files in dest not in source
`-exclude=`	Exclude files matching pattern
`-e ssh`	Use SSH for remote transfers
`-backup`	Make backups of overwritten files

Examples

```
# Basic local sync
$ rsync -av source/ destination/

# IMPORTANT: Trailing slash matters!
# source/  = contents of source directory
# source   = the source directory itself

# Sync with progress
$ rsync -avP source/ destination/
$ rsync -av --progress source/ destination/

# Dry run (test without changes)
$ rsync -avn source/ destination/
```

```
# Mirror directories (delete extra files in dest)
$ rsync -av --delete source/ destination/

# Exclude files
$ rsync -av --exclude="*.log" --exclude=".git" source/ dest/

# Sync to remote server via SSH
$ rsync -avz /local/folder/ user@remote:/remote/folder/

# Sync from remote server
$ rsync -avz user@remote:/remote/folder/ /local/folder/

# Sync with specific SSH port
$ rsync -avz -e "ssh -p 2222" source/ user@remote:/dest/

# Backup with date folder
$ rsync -av /home/user/ /backup/$(date +%Y%m%d)/

# Incremental backup with hard links (space efficient)
$ rsync -av --link-dest=/backup/yesterday /home/user/ /backup/today/
```

cpio - Copy In/Out Archive

Examples

```
# Create archive from file list
$ find . -name "*.txt" | cpio -ov > archive.cpio

# Create archive (newer format)
$ find . -name "*.conf" | cpio -ov -H newc > archive.cpio

# Extract archive
$ cpio -idv < archive.cpio

# Copy files to another directory
$ find . -name "*.log" | cpio -pdv /backup/logs/

# List contents
$ cpio -itv < archive.cpio

# Common flags:
# -o = create (output)
# -i = extract (input)
# -p = pass-through (copy)
# -v = verbose
# -d = create directories as needed
```

dd - Data Duplicator

Warning

dd is DANGEROUS!
A typo in `if=` or `of=` can destroy your entire system! The command is sometimes called "disk destroyer." Always:

- Double-check source (if) and destination (of)
- Use `lsblk` to verify device names
- Never run dd commands you don't fully understand

Flags and Options

`if=FILE`	Input file (source)
`of=FILE`	Output file (destination)
`bs=SIZE`	Block size (e.g., 4M, 1G)
`count=N`	Copy only N blocks
`status=progress`	Show progress
`conv=sync`	Pad blocks with zeros
`conv=noerror`	Continue on read errors

Examples

```
# Create disk image
$ sudo dd if=/dev/sda of=/backup/disk.img bs=4M status=progress

# Write ISO to USB (CREATE BOOTABLE USB)
$ sudo dd if=ubuntu.iso of=/dev/sdb bs=4M status=progress

# Clone disk to disk
$ sudo dd if=/dev/sda of=/dev/sdc bs=4M status=progress

# Create file of specific size (for testing)
$ dd if=/dev/zero of=testfile bs=1M count=100
# Creates 100MB file filled with zeros

# Create random data file
$ dd if=/dev/urandom of=random.bin bs=1M count=10

# Backup MBR (first 512 bytes)
$ sudo dd if=/dev/sda of=mbr.bak bs=512 count=1

# Restore MBR
$ sudo dd if=mbr.bak of=/dev/sda bs=512 count=1

# Wipe first MB of disk (VERY DANGEROUS!)
$ sudo dd if=/dev/zero of=/dev/sda bs=1M count=1
```

Hands-On Practice

Day 14 Exercises:
Exercise 1: Create and Extract Archives

```
$ mkdir test_backup && cd test_backup
$ echo "test1" > file1.txt
```

```
$ echo "test2" > file2.txt
$ mkdir subdir && echo "test3" > subdir/file3.txt

# Create compressed archive
$ tar -czvf backup.tar.gz .

# List contents
$ tar -tzvf backup.tar.gz

# Extract to new directory
$ mkdir restore
$ tar -xzvf backup.tar.gz -C restore/
$ ls -la restore/
```

Exercise 2: Compare Compression

```
$ dd if=/dev/zero of=testfile bs=1M count=10
$ cp testfile testfile.gzip && gzip testfile.gzip
$ cp testfile testfile.bzip && bzip2 testfile.bzip
$ cp testfile testfile.xz && xz testfile.xz
$ ls -lh testfile*
```

Exercise 3: Practice rsync

```
$ mkdir source dest
$ touch source/{a,b,c}.txt
$ rsync -av source/ dest/
$ ls dest/
$ touch source/d.txt
$ rsync -av source/ dest/  # Only d.txt transferred
```

Interview Question

Q: How do you create a compressed backup of a directory?
Use `tar -czvf backup.tar.gz /path/to/directory` to create a gzip-compressed archive. Use `-cjvf` for bzip2 or `-cJvf` for xz compression.

Q: What is the difference between tar and rsync for backups?
`tar` creates archive files (one file containing many). `rsync` synchronizes directories by copying only changed files. rsync is better for incremental backups and remote sync.

Q: How do you create a bootable USB drive from an ISO?
Use `sudo dd if=image.iso of=/dev/sdX bs=4M status=progress` where sdX is the USB device. Be VERY careful to specify the correct device.

DAY 15: Linux Networking Basics

IP Addresses, Interfaces, and Connectivity

Theory

Networking Fundamentals:

- **IP Address:** Unique identifier for a device on a network (e.g., 192.168.1.100)
- **MAC Address:** Physical hardware address (e.g., 08:00:27:12:34:56)
- **Network Interface:** Connection point (eth0, ens33, wlan0)
- **Gateway:** Router that connects to other networks
- **Subnet Mask:** Defines network size (255.255.255.0 or /24)
- **DNS:** Translates domain names to IP addresses

Common Interface Names:

- `lo` - Loopback (127.0.0.1)
- `eth0` - First Ethernet (old naming)
- `ens33, enp0s3` - Ethernet (new predictable naming)
- `wlan0, wlp2s0` - Wireless
- `docker0` - Docker bridge network
- `virbr0` - Virtual bridge (libvirt)

ifconfig - Interface Configuration (Legacy)

Theory

`ifconfig` is the traditional tool for network configuration. It's deprecated but still widely used. On newer systems, you may need to install it:
`sudo apt install net-tools` (Ubuntu/Debian)
`sudo yum install net-tools` (CentOS/RHEL)
The modern replacement is the `ip` command.

Flags and Options

`ifconfig`	Show all active interfaces
`ifconfig -a`	Show ALL interfaces (including inactive)
`ifconfig IFACE`	Show specific interface
`ifconfig IFACE up`	Bring interface up
`ifconfig IFACE down`	Bring interface down
`ifconfig IFACE IP`	Set IP address (temporary)
`ifconfig IFACE netmask MASK`	Set netmask
`ifconfig IFACE hw ether MAC`	Set MAC address

Examples

```
# Show all active interfaces
$ ifconfig
```

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.100  netmask 255.255.255.0  broadcast
           192.168.1.255
        inet6 fe80::a00:27ff:fe12:3456  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:12:34:56  txqueuelen 1000  (Ethernet)
        RX packets 12345  bytes 1234567 (1.2 MB)
        TX packets 6789  bytes 567890 (554.6 KB)

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 1234  bytes 123456 (120.5 KB)
        TX packets 1234  bytes 123456 (120.5 KB)

# Show all interfaces (including inactive)
$ ifconfig -a

# Show specific interface
$ ifconfig eth0

# Bring interface down then up
$ sudo ifconfig eth0 down
$ sudo ifconfig eth0 up

# Set IP address (temporary - lost on reboot)
$ sudo ifconfig eth0 192.168.1.50 netmask 255.255.255.0
```

ip addr - Show/Manipulate IP Addresses (Modern)

Flags and Options

`ip addr` or `ip a`	Show all addresses
`ip addr show dev IFACE`	Show specific interface
`ip -4 addr`	Show IPv4 only
`ip -6 addr`	Show IPv6 only
`ip addr add IP/MASK dev IFACE`	Add IP address
`ip addr del IP/MASK dev IFACE`	Delete IP address
`ip addr flush dev IFACE`	Remove all addresses

Examples

```
# Show all IP addresses (most common command)
$ ip addr
$ ip a    # Short form

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
```

```
       valid_lft forever preferred_lft forever

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether 08:00:27:12:34:56 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe12:3456/64 scope link
       valid_lft forever preferred_lft forever

# Show specific interface
$ ip addr show dev eth0
$ ip a s eth0    # Short form

# Show only IPv4
$ ip -4 addr

# Show only IPv6
$ ip -6 addr

# Add IP address (temporary)
$ sudo ip addr add 192.168.1.50/24 dev eth0

# Add secondary IP address
$ sudo ip addr add 192.168.1.51/24 dev eth0

# Delete IP address
$ sudo ip addr del 192.168.1.50/24 dev eth0

# Show addresses in brief format
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             192.168.1.100/24 fe80::a00:27ff:fe12
    :3456/64
```

ip link - Network Interface Control

Flags and Options

`ip link` or `ip l`	Show all interfaces
`ip link show dev IFACE`	Show specific interface
`ip -s link`	Show with statistics
`ip link set IFACE up`	Bring interface up
`ip link set IFACE down`	Bring interface down
`ip link set IFACE mtu SIZE`	Set MTU
`ip link set IFACE address MAC`	Set MAC address
`ip link set IFACE promisc on`	Enable promiscuous mode

Examples

```
# Show all network interfaces
$ ip link
$ ip l    # Short form

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
    link/ether 08:00:27:12:34:56 brd ff:ff:ff:ff:ff:ff

# Show with statistics (packets, bytes, errors)
$ ip -s link
$ ip -s link show dev eth0

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    RX: bytes  packets  errors  dropped overrun mcast
    1234567    12345    0       0       0       0
    TX: bytes  packets  errors  dropped carrier collsns
    567890     6789     0       0       0       0

# Brief format
$ ip -br link
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0             UP             08:00:27:12:34:56 <BROADCAST,MULTICAST,UP
   >

# Bring interface up/down
$ sudo ip link set eth0 down
$ sudo ip link set eth0 up

# Change MTU
$ sudo ip link set eth0 mtu 9000

# Change MAC address (requires interface down)
$ sudo ip link set eth0 down
$ sudo ip link set eth0 address 00:11:22:33:44:55
$ sudo ip link set eth0 up
```

ip route - Routing Table

Flags and Options

`ip route` or `ip r`	Show routing table
`ip route show`	Same as above
`ip route get IP`	Get route to specific IP
`ip route add NETWORK via GW`	Add static route
`ip route del NETWORK`	Delete route
`ip route add default via GW`	Set default gateway
`ip route flush cache`	Flush routing cache

Examples

```
# Show routing table
$ ip route
$ ip r     # Short form

default via 192.168.1.1 dev eth0 proto static metric 100
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100 metric
    100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1

# Explanation:
# default via 192.168.1.1 = Default gateway is 192.168.1.1
# 192.168.1.0/24 dev eth0 = Local network via eth0

# Get route to specific IP
$ ip route get 8.8.8.8
8.8.8.8 via 192.168.1.1 dev eth0 src 192.168.1.100 uid 1000

$ ip route get 192.168.1.50
192.168.1.50 dev eth0 src 192.168.1.100 uid 1000
    cache

# Add static route
$ sudo ip route add 10.0.0.0/8 via 192.168.1.254

# Delete route
$ sudo ip route del 10.0.0.0/8

# Change default gateway
$ sudo ip route del default
$ sudo ip route add default via 192.168.1.1

# Show routing table with details
$ ip route show table all
```

ping - Test Network Connectivity

Flags and Options

`-c COUNT`	Stop after sending COUNT packets
`-i INTERVAL`	Seconds between packets (default 1)
`-W TIMEOUT`	Timeout waiting for reply (seconds)
`-s SIZE`	Packet size in bytes (default 56)
`-q`	Quiet (only show summary)
`-f`	Flood ping (fast, requires root)
`-4`	Use IPv4 only
`-6`	Use IPv6 only
`-I IFACE`	Use specific interface

Examples

```
# Basic ping (Ctrl+C to stop)
$ ping google.com
PING google.com (142.250.67.206) 56(84) bytes of data.
64 bytes from 142.250.67.206: icmp_seq=1 ttl=117 time=15.2 ms
64 bytes from 142.250.67.206: icmp_seq=2 ttl=117 time=14.8 ms
^C
--- google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 14.823/15.011/15.200/0.188 ms

# Send specific number of packets
$ ping -c 4 google.com

# Ping with shorter interval
$ ping -c 10 -i 0.2 google.com

# Ping with larger packet
$ ping -c 4 -s 1000 google.com

# Quiet mode (summary only)
$ ping -c 4 -q google.com
PING google.com (142.250.67.206) 56(84) bytes of data.
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 14.523/15.011/15.600/0.388 ms

# Ping through specific interface
$ ping -c 4 -I eth0 8.8.8.8

# Test if host is reachable (script friendly)
$ ping -c 1 -W 2 google.com > /dev/null && echo "UP" || echo "DOWN"
```

Pro Tip

Network Troubleshooting Steps:

1. `ping 127.0.0.1` - Test loopback (TCP/IP stack working?)
2. `ping YOUR_IP` - Test local interface
3. `ping GATEWAY_IP` - Test connection to router
4. `ping 8.8.8.8` - Test internet connectivity (Google DNS)
5. `ping google.com` - Test DNS resolution

If step 4 works but step 5 fails = DNS problem!
Check `/etc/resolv.conf` for DNS servers.

Hands-On Practice

Day 15 Exercises:
Exercise 1: View Network Configuration

```
$ ip addr
$ ip link
$ ip route
$ cat /etc/resolv.conf
```

Exercise 2: Extract Specific Information

```
# Find your IP address
$ ip addr show | grep "inet " | grep -v 127.0.0.1

# Find your default gateway
$ ip route | grep default

# Find your MAC address
$ ip link show eth0 | grep ether
```

Exercise 3: Test Connectivity

```
$ ping -c 2 127.0.0.1         # Loopback
$ ping -c 2 $(ip route | grep default | awk '{print $3}')  # Gateway
$ ping -c 2 8.8.8.8           # Internet
$ ping -c 2 google.com        # DNS
```

Exercise 4: Check Interface Statistics

```
$ ip -s link show eth0
```

Interview Question

Q: What is the difference between ifconfig and ip command?
`ifconfig` is the legacy command from net-tools package. `ip` is the modern replacement that is more powerful and is part of iproute2 package. Always prefer `ip` on modern systems.

Q: How do you find your default gateway?
Use `ip route | grep default` or `ip r` and look for the line starting with "default via".

Q: How do you troubleshoot network connectivity issues?
Start by pinging localhost (127.0.0.1), then your own IP, then gateway, then external IP (8.8.8.8), then domain name. This helps identify whether the issue is with local stack, LAN, internet, or DNS.

Day 11-15 Quick Reference

Command	Purpose	Example
Package Management (Debian)		
apt-get update	Update package list	sudo apt-get update
apt-get upgrade	Upgrade packages	sudo apt-get upgrade -y
apt-get install	Install package	sudo apt-get install nginx
apt-get remove	Remove package	sudo apt-get remove nginx
apt-cache search	Search packages	apt-cache search nginx
dpkg -l	List installed	dpkg -l | grep nginx
dpkg -S	Find package owner	dpkg -S /usr/bin/vim
Package Management (RHEL)		
yum install	Install package	sudo yum install httpd
yum update	Update packages	sudo yum update
yum remove	Remove package	sudo yum remove httpd
rpm -qa	List installed	rpm -qa | grep http
rpm -qf	Find package owner	rpm -qf /usr/sbin/httpd
Services		
systemctl start	Start service	sudo systemctl start nginx
systemctl stop	Stop service	sudo systemctl stop nginx
systemctl restart	Restart service	sudo systemctl restart nginx
systemctl enable	Enable at boot	sudo systemctl enable nginx
systemctl status	Check status	systemctl status nginx
System Info		
uname -a	System info	uname -a
hostname -I	IP addresses	hostname -I
uptime	Uptime and load	uptime
free -h	Memory usage	free -h
lscpu	CPU info	lscpu
dmesg	Kernel messages	dmesg | tail
Backup and Archiving		
tar -czvf	Create .tar.gz	tar -czvf backup.tar.gz dir/
tar -xzvf	Extract .tar.gz	tar -xzvf backup.tar.gz
tar -tzvf	List contents	tar -tzvf backup.tar.gz
rsync -av	Sync directories	rsync -av src/ dest/
zip -r	Create zip	zip -r archive.zip dir/
unzip	Extract zip	unzip archive.zip
Networking		

Command	Purpose	Example
ip addr	Show IP addresses	ip a
ip link	Show interfaces	ip l
ip route	Show routing	ip r
ping	Test connectivity	ping -c 4 google.com
ifconfig	Legacy IP config	ifconfig

Day 11-15 Complete!

System Administration Fundamentals Mastered

Prepared by Raman

#Linux #DevOps #SysAdmin